

Your Second Brain

Persistent memory, tools and cumulative knowledge

Agentic AI for Economic Research | Lesson 2 | Marco Le Moglie

GitHub repo

What JARVIS did, an agent can do



The film

Tony designs a suit with JARVIS. Iron Man 3.



The same idea, built

A JARVIS-style assistant, made with an AI agent.

What was science fiction is now possible with AI agents.

Why JARVIS is not just a chatbot

A chatbot answers questions. JARVIS lives inside Tony Stark's workspace: it knows his projects, uses his machines and asks before acting.

Four features make the difference.

Memory

Remembers projects, past choices, constraints and an evolving work history.

Tools

Reads files and interacts with authorized applications and systems.

Context

Knows which project, artifact and stage of work matters now.

Action

Executes constrained workflows, reports progress and asks for approval.

Chatbot → agent → agent with a Brain

A Second Brain is an infrastructure outside the harness.

- **Chatbot.** Answers only from what is in the current conversation: your messages and the files you paste in.
- **Agent.** Can plan tool calls, read files and change the environment within permissions.
- **Persistent Brain.** Adds durable project state, approved decisions, sources, skills and working procedures.
- **Control loop.** Human review, version history and a closeout protocol prevent invisible drift.

Assistant quality = model capability + relevant memory + tools + controls.

The problem: every session starts over

A six-month research project cannot be reduced to one long chat.

- **Reconstruction cost.** Re-explain the question, dataset, code locations, methodological choices and pending work.
- **Memory failures.** Earlier caveats or rejected ideas disappear from the model's accessible context.
- **Context rot.** Long conversations can bury important facts among irrelevant traces and repeated instructions.
- **Coordination cost.** Coauthors and new agents must infer what the team decided and why.

Without a Brain

Chat history + folders + implicit human memory.

With a Brain

Stable restart point, selective retrieval and explicit decision history.

A durable project needs a state representation, not only a transcript.

What should the Brain remember?

Persist the knowledge required to make the next session productive.

Project state

Question, progress, current bottleneck, data and code status.

Decisions

Accepted methodological rules, rejected alternatives and rationale.

Evidence

Sources, original data pointers, confidence, provenance and uncertainty.

Procedures

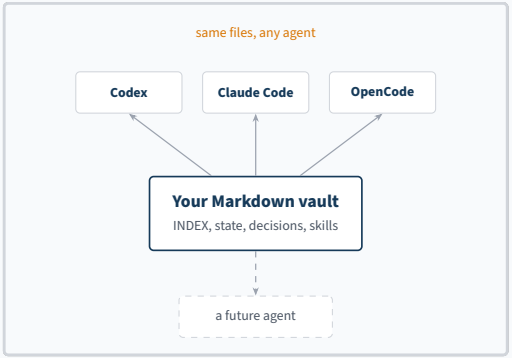
Commands, validation routines, reusable prompts and skills.

Do not promote every chat message: forget noise, retain durable work.

Own your knowledge, not your model

One external vault can be read by different agents when access and instructions are configured.

- **Open representation.** Markdown + optional YAML/JSON metadata, readable in any editor.
- **Portability.** Use the same project state from Codex, Claude Code, OpenCode or a future agent.
- **One source of truth.** Avoid divergent copies of facts in proprietary AI profiles or prompts.
- **Qualification.** Skills, permissions, connectors and model behavior are not automatically portable.



New session, same project knowledge

Reset the conversation when useful; do not reset the project.

- **Before closeout.** Update the current state, next actions, decisions and validated technical notes.
- **At startup.** Read global routing instructions; identify the project; load only relevant core files.
- **In the new session.** Recover the task constraints without importing the entire previous chat.

Short sessions avoid context rot

Because the state is saved, you can close a session and open a new one often. Each session stays short, so old material never piles up and drowns the facts that matter.

Load only what the task needs

The new session reads a small index first, then opens only the one or two project files the task requires.

A good closeout is what makes a clean start possible.

Less context can mean fewer tokens

A worked example: answering a referee on a staggered difference-in-differences paper. Token counts are rough orders of magnitude.

- **The task.** Referee 2 asks for a pre-trend test. You ask the agent to draft the new event-study paragraph and the reply.
- **Everything in the prompt.** Full draft (25,000 tokens) + ten related papers (60,000) + do-files and logs (15,000): about **100,000 tokens**, sent again at every request.
- **From the Brain.** CURRENT_STATE and DECISIONS (2,000) + the referee comment and Section 4 (4,000) + the event-study log (2,000): about **8,000 tokens**, roughly 92% fewer.

Optimize relevant information delivered per token, not token count alone.

Portable skills: instructions on demand

A file-based skill transfers a tested workflow from one session or agent to another.

- **Catalog.** Name and description tell an agent that a skill exists.
- **Activation.** The full SKILL.md procedure loads when the task calls for it.
- **Resources.** Scripts, examples and references load only when needed.

Your implementation

Cross-project skills in general/capabilities/skills; agent adapters in .agents/skills.

What it costs

At every start the agent reads the list of skill names and one-line descriptions, a few hundred tokens. The full steps of a skill enter the context only when the agent opens it.

Store once, discover cheaply, load selectively.

From chatbot to research assistant

A paper is a chain of decisions and dependencies, not a sequence of chat messages.

- **Research question.** What is the mechanism, population and contribution?
- **Data design.** How were sources collected, matched, cleaned and validated?
- **Identification.** Which assumptions, timing choices and robustness checks are binding?
- **Research continuity.** How does the next agent recover a reproducible restart point?

Economics insight

Memory can lower re-entry and coordination costs for distributed research teams.

Research integrity

An agent should cite evidence and preserve unresolved uncertainty.

The value is a cumulative and auditable work, thus a better answer.

A short history of external memory

Different traditions solve different parts of the memory problem.

- **1945.** Vannevar Bush imagined the Memex: a desk that stores a scientist's books, papers and notes on microfilm and lets him save chains of links between them. It was never built.
- **2000s–2020s.** Personal knowledge management: real note-taking software with links between notes (Evernote, Roam, Obsidian), plus methods for filing notes, such as Tiago Forte's Projects, Areas, Resources, Archives (PARA). The person still does all the linking.
- **2010s–2020s.** Search, embeddings, retrieval-augmented generation and knowledge graphs.
- **2024–2026.** The model stops only fetching documents and starts writing: an agent keeps an up-to-date summary of what the documents say (GraphRAG summaries, Karpathy's LLM Wiki).

Source: Bush (1945); Forte (2022); Lewis et al. (2020); Edge et al. (2024); Karpathy (2026)

Avoid a false ladder

RAG, wiki, graphs and notes coexist; no single approach makes the others obsolete.

Today's question

What persists across tasks: raw documents, distilled knowledge, decisions or executable worksflows?

A Brain architecture is a choice about what is stored, retrieved and updated.

Approach 1: put everything in the prompt

The simplest external-memory strategy is often the least selective.

- **Design.** Attach notes, instructions and all source material to each new request.
- **Strength.** No retrieval stack, transparent inputs and straightforward debugging.
- **Weakness.** Large inputs, repeated material and often low signal density.
- **Best use.** Small and stable corpora for a narrow and high-stakes task where full context fits.

Common mistake

Assuming that if the documents fit in the context window, the model reads all of them with equal care. Facts buried in the middle of a long input are recalled less often.

Improvement

Move from full read to indexed, evidence-backed selective disclosure.

A larger window allow you to paste more, but it does not say which decision is current, which idea was rejected, or who approved a change.

Approach 2: Retrieval-Augmented Generation (RAG

RAG retrieves source evidence when a question arrives.

- **Prepare.** Parse documents, split or structure text and build searchable indexes.
- **Query.** Retrieve relevant passages using lexical search, vector similarity or both.
- **Generate.** Answer with retrieved context and citations to original documents.
- **Risks**:
  - **Missed evidence.** Search never returns the appendix table, so the answer goes on without it.
  - **Bad chunk boundaries.** A coefficient is cut off from the sentence that says which regression it belongs to.
  - **Ambiguous provenance.** Two notes disagree, and the answer does not say which file it used.
  - **Stale index.** The paper changed last week; search still returns last month's draft.

RAG finds the passages in your documents that match a question. It does not keep track of where the project stands: current decisions, rejected ideas, next steps.

Approach 3: Knowledge bases, links and GraphRAG

An Obsidian link only says that two notes are related.
A knowledge graph also says how: this paper uses that dataset, this result contradicts that one.

- **Wiki and backlinks.** Human-readable notes form navigable relationships.
- **Typed graph.** An edge can mean uses-method, produced-by or contradicts.
- **GraphRAG.** Graph-based summaries can assist questions spanning many documents.
- **Trade-off.** More expressive relationships require extraction, validation and maintenance.

Economics graph

Paper → identification → dataset → do-file → figure → reported effect.

Practical starting point

First maintain linked Markdown; add typed relations only when necessary.

Approach 3.1: A LLM-maintained wiki

In April 2026 Andrej Karpathy (OpenAI, Anthropic) proposed that the agent write and keep up to date a wiki of what your sources say, so the next question starts from that summary instead of from the raw files.

- **Raw.** Immutable or preserved source materials: papers, web pages, transcripts.
- **Wiki.** Interlinked, editable Markdown pages synthesizing what sources establish.
- **Schema.** Agent-facing rules for naming, links, ingestion and maintenance.
- **Compounding effect.** New sources update old pages; useful answers may be filed back into the wiki.

The agent is the maintainer; Obsidian is a readable interface.

LLM Wiki workflow: ingest, query, lint

The cycle runs in this order: ingest new sources, query the wiki, lint it from time to time, promote what proved useful.

1. INGEST

Preserve source; identify claims and entities; patch affected pages; preserve citations.

2. QUERY

Read index; follow relevant pages; recover supporting sources; synthesize a reply.

3. LINT

Check broken links, orphan notes, duplications, contradictions and stale claims.

4. PROMOTE

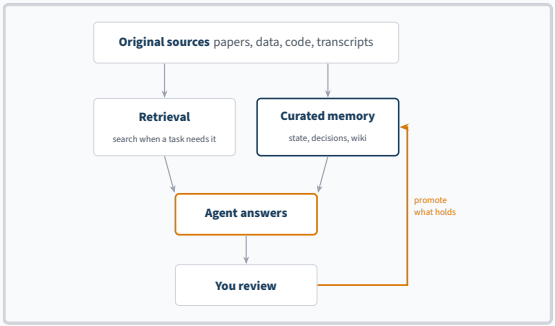
Save vetted synthesis or a reusable method for future tasks.

Record uncertain or contradictory evidence; do not silently replace it.

An effective hybrid: RAG + curated wiki

Use one layer for detailed evidence and another for stable distilled knowledge.

- **Original sources.** Keep papers, code, transcripts and records available for verification.
- **Retrieval.** Search sparse, situational evidence when the task demands it.
- **Curated memory.** Maintain dense, frequently used rules, positions, assumptions and project state.
- **Governance.** Validate edits against source evidence before promoting them to canonical memory.



The retrieval layer complements memory; it does not replace it.

Which architecture should you choose?

It depends on how much source material you have and how long the project lasts.

- **Prompt-only.** Small, finite task: minimal infrastructure, higher repetition cost.
- **RAG.** Large reference corpus: stronger source discovery and evidence retrieval.
- **LLM Wiki.** Repeated synthesis within a continuing domain: curated compounding knowledge.
- **Hybrid.** Long-running research or enterprise work: state + wiki + targeted retrieval.

Rule of thumb

A few dozen pages: paste them. Hundreds of documents: add search. A project that runs for months: keep files with its state and decisions.

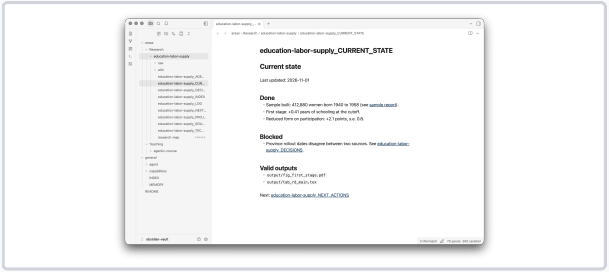
In this course

We build the hybrid: Markdown project files, plus search when the sources get large.

Why Obsidian?

Obsidian edits an ordinary folder of Markdown files, rather than locking knowledge into an AI profile.

- **Local files.** A vault is a directory; notes are human-readable .md files.
- **Connections.** Links, backlinks, search and properties organize knowledge.
- **Extensions.** Templates, Canvas and plugins add convenience.
- **Important.** Obsidian is the editor; agents and protocols create the AI behavior.



The lesson vault, a fake project on education and labour supply. Folder: lesson2/obsidian-vault.

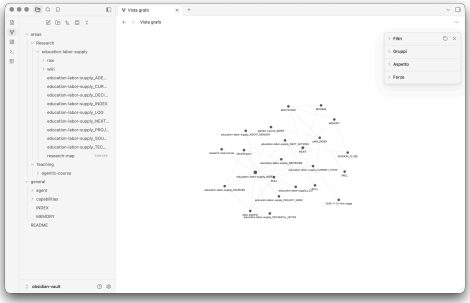
Vaults, folders and plain Markdown

Folders tell you where a file lives. A fixed set of files in every project (brief, state, decisions, next actions) tells the next session where the work stands.

- **Vault.** Top-level folder, openable in Obsidian and another editor.
- **Indexes.** Explicit maps tell readers and agents where project content lives.
- **Metadata.** Markdown frontmatter can record type, owner, status and provenance.
- **No requirement.** You do not need vector embeddings to create a useful first Brain.

Links, backlinks and the Graph View

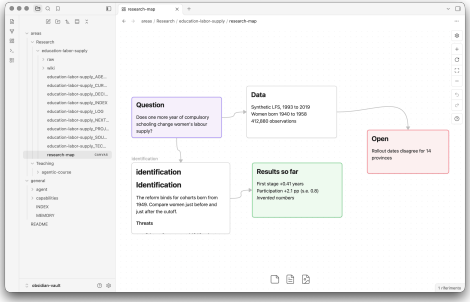
Obsidian can reveal connections across notes and projects.



The graph shows that two notes are linked. It cannot tell you whether the link makes sense or whether the note is still true: someone has to check.

Canvas: think spatially

A flexible surface for questions, evidence, methods and results.



Use Canvas to sketch. Once a choice is made, write it in DECISIONS.md, where the next session will look for it.

Your first Brain: four useful project files

Start with a small folder; let complexity emerge from recurring needs.

- **PROJECT_BRIEF.** What we are doing, why it matters, assumptions and scope.
- **CURRENT_STATE.** What is complete, what is blocked, which outputs are valid.
- **NEXT_ACTIONS.** Concrete, prioritized tasks that allow a cold restart.
- **DECISIONS.** What is agreed, why, with dates and consequences.

Add one index

general/INDEX.md points to active projects and global rules.

Grow on demand

SOURCES, AGENT_MEMORY, LOG, wiki/ and skills/ only as real needs emerge.

Connect an AI agent safely

The agent needs explicit filesystem access and a repeatable entry point.

- **Workspace.** Open the vault folder or provide the path using the agent's supported tools.
- **Entry point.** Give BOOTSTRAP.md: where to read, how to route, what not to touch.
- **Permission scope.** Read-only for initial onboarding; writes require clear approval and Git diffs.
- **Session protocol.** Start from index and state; end by updating validated durable memory.

Terminal agents

Start Codex, Claude Code or OpenCode from inside the vault folder: `cd obsidian-vault` and then `codex`.

Cursor

File, Open Folder, and pick the vault. The agent can now read every note in it.

Reading the vault is not remembering. The agent uses the Brain only if BOOTSTRAP.md tells it which files to open at the start and which to update at the end.

Let AI build the first version

Scaffolding a Brain is a small task, and the result is easy to test: open a new session and ask the agent where the project stands.

- **Provide constraints.** Work areas, privacy rules, naming, project file roles and desired workflows.
- **Generate.** Index, template files, bootstrap, closeout and minimal validator.
- **Review.** Inspect the folder tree, check descriptions and open the vault in Obsidian.
- **Iterate.** Use an actual project for a week before adding complex memory backends.

Simple prompt

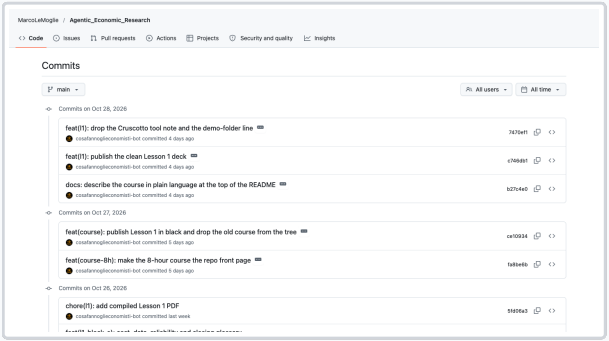
“Create a Markdown Brain with global index, project state, decisions and closeout; do not overwrite existing files.”

AI can create the structure; humans approve its memory policy.

Git + GitHub: version the knowledge

Dropbox, Git and GitHub perform complementary jobs.

- **Dropbox.** Synchronizes the same files across devices; not a substitute for reviewed commits.
- **Git.** Records versions, shows diffs and allows local rollback.
- **GitHub.** Optional remote host for collaboration and review; private by default for sensitive work.
- **Security.** Exclude credentials; review changes before push; handle concurrent edits.



The commit history of this course on GitHub: every change has a date, a message and a diff.

My Brain: three layers of one system

Three top-level folders, each with one job: shared rules, one folder per project, and hidden tooling.

- **general/.** Shared indexes, global decisions, profiles, tool policies and agent protocols.
- **areas/.** Research, Grant, Teaching and Other; a self-contained memory per project.
- **.infra/.** Hidden scripts and tool runtimes; not canonical project knowledge.
- **Source of truth.** Markdown in general/ and areas/; other backends are supporting layers.

Real vault

Brain/Marco on Dropbox; Desktop alias points to the same folder.

Core principle

Global routing first; load one project's memory, not every project.

One durable structure, multiple agents and many research activities.

One Brain, many Economics projects

Each project has its own context and methodological decisions.

- **Mafia Culture.** Artist, work, recording and video distinctions; verified sourcing and genre classification.
- **Fragmented Opposition.** Repeatable news-collection workflow with failed-only retries and deterministic merges.
- **Mafia Incendi.** Research code and Stata folders; unresolved questions remain visibly flagged.
- **Other areas.** Grants, teaching and governance of the Brain itself.

Cross-project memory

A skill moves to the global catalog only when it is reusable for different projects.

The Brain preserves why decisions were made, not only the latest output.

Anatomy of a research project

Different kinds of information have distinct filenames and lifecycles.

- **INDEX / PROJECT_BRIEF.** Navigation and stable problem definition.
- **CURRENT_STATE / NEXT_ACTIONS.** Current progress and actionable restart point.
- **DECISIONS / AGENT_MEMORY.** Approved choices and durable constraints for future agents.
- **SOURCES / TECHNICAL_NOTES / LOG.** Provenance, commands and historical trace; optional wiki and session summaries.

Why one file per question?

Each file answers one question: where are we, what did we decide, what comes next. To pick up the analysis, the agent opens CURRENT_STATE and NEXT_ACTIONS and skips the rest.

Human benefit

A new coauthor can see what is known, blocked and contested.

Project memory = state + evidence + decisions + procedures.

The tool stack around the Brain

The vault is the center; individual tools add functions.

- **Authoring / sync.** Obsidian, Markdown, Dropbox and Git.
- **Recall / access.** Memanto (a local memory the agent can search).
- **Code context / compression.** CodeGraph (a map of the code base, so the agent finds functions without reading every file) and Headroom (compresses long tool outputs).
- **Procedures.** Caveman, Econoclast, anti-slop-prose and Overleaf workflow.

Skills live in the Brain, not one AI

- **Global procedures.** general/capabilities/skills/ holds reusable cross-project workflows.
- **Local procedures.** areas/.../skills/ keeps assumptions close to a specific project.
- **Native exposure.** .agents/skills/ makes selected skills available to compatible agents.
- **Promotion rule.** Only tested recurring routines become global skills.

Catalog cheaply; load instructions only when a task needs them.

My LLM Wiki: core files first

Karpathy's cumulative wiki adapted to project-oriented scientific work.

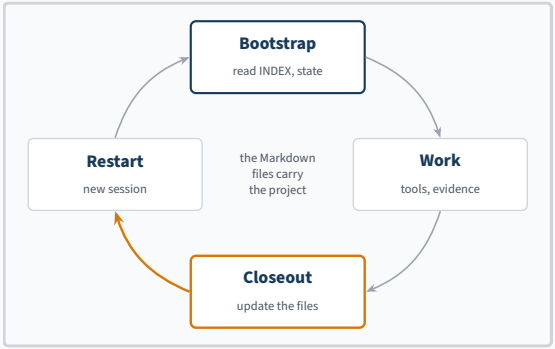
- **Preserve.** Original conversations and source traces under the relevant project's raw/ directory.
- **Distill.** Update CURRENT_STATE, DECISIONS, AGENT_MEMORY and TECHNICAL_NOTES first.
- **Extend.** Use wiki/ for stable long-form pipelines, methods and architectures.
- **Index.** Every wiki page must appear under the project INDEX's Wiki section.

Update the short core files first. Write a long wiki page only for material that will not change often, such as a data pipeline.

Bootstrap → work → closeout → restart

A reproducible routine makes independent sessions part of one project.

- **Bootstrap.** Global INDEX and memory router → specific project files → skills only if relevant.
- **Work.** Run tools and check evidence, errors, current code and open assumptions.
- **Closeout.** Update state, next actions, decisions, technical notes and concise session log.
- **Restart.** A new agent recovers a compact, trustworthy entry point rather than old chat context.



The next session should open the state file and start working, not dig through old chats to find out what happened.

A Brain also accumulates maintenance debt

More files are not necessarily more useful knowledge.

Stale state

Completed tasks still shown as next actions; paths point to old locations.

Broken navigation

Orphan wiki pages, missing references and outdated indexes.

Contradictions

Competing claims or decisions, with no explicit resolution history.

Duplicate procedures

Slightly divergent skills and templates across projects or agents.

Maintenance is part of the memory architecture.

My Sunday Brain maintenance: a real audit

1. Read BOOTSTRAP and the vault index.
2. Commit and push a Git snapshot; scan file names for secrets.
3. Scan every note; ingest the week's raw chats into the core files.
4. Align each project's state; move finished actions to the LOG.
5. Update the projects dashboard and the dates in MEMORY.md.
6. Fix dead links and paths that point to old locations.
7. Check that every wiki page is listed in its project INDEX.
8. Delete duplicates and backups, recoverable from the snapshot.
9. Look for sensitive data (keys, IBAN, booking codes).
10. Check that Memanto runs locally; run validate_vault.py.
11. Look for new plugins and tools worth adding.
12. Update the plugins and tools already installed.
13. Write the LOG and a closeout; ask me open questions.

Keep humans in control of the Brain

The agent does the maintenance; I review what it changed before the change becomes permanent.

- **Every session.** Distill durable findings; mark uncertainty and next action.
- **Every review.** Check stale state, index completeness, contradictory memory and orphan files.
- **Before changes.** Snapshot with Git, inspect diffs, exclude secrets and preserve evidence.
- **After changes.** Validate and record what actually changed; test new-agent restart.

Research payoff

Lower re-entry cost, portable expertise, potentially stronger reproducibility.

Open question

Does persistent curated memory outperform pure chat and RAG on longitudinal research tasks? Test it.

JARVIS depends not only on a model, but on reliable memory, tools and governance.

Appendix: important sources

Core readings students can use to construct or evaluate a Brain.

- **Karpathy (2026).** LLM Wiki idea file: raw sources, wiki, schema, ingest/query/lint.
- **Lewis et al. (2020).** Retrieval-Augmented Generation: source retrieval at inference.
- **Edge et al. (2024).** GraphRAG for synthesis across many documents.

Practice

Obsidian Help; Agent Skills specification; Git documentation.

Economics

Walsh/Ungson (1991); Cohen/Levinthal (1990); Brynjolfsson et al. (2025).

Appendix: making the live demo safe

Continue from Lesson 1: the short scripts, then Sportello Italia, then the Brain around that project.

- **Open the Brain.** Show the real folder: shared rules, one folder per project, and how that structure was built.
- **Ingest.** File the Sportello Italia notes into that project: state, decisions, sources.
- **Reopen.** Start a new session on the project and walk through the parts the agent actually reads.
- **Close.** Run the closeout prompt: what changed, what is next, what stays out of the files.
- **Overleaf, Dropbox, the phone, ChatGPT.** The project folder is on Dropbox, so all four can see it. Show the join, then a file landing in Dropbox.
- **What else fits.** Obsidian plugins extend the vault. The graph is one example.

References

Concepts and architectures

Bush, V. (1945). As we may think. *The Atlantic*, July. theatlantic.com.

Forte, T. (2022). *Building a Second Brain*. Atria Books. buildingasecondbrain.com.

Lewis, P. et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *NeurIPS*. arXiv:2005.11401.

Edge, D. et al. (2024). From local to global: a Graph RAG approach to query-focused summarization. arXiv:2404.16130.

Karpathy, A. (2026). LLM Wiki. GitHub Gist, 4 April. gist.github.com/karpathy.

Anthropic (2024). Building effective agents. anthropic.com/engineering.

Anthropic. Effective context engineering for AI agents. anthropic.com/engineering.

Agent Skills specification. agentskills.io/specification.

Economics of knowledge and memory

Hayek, F. A. (1945). The use of knowledge in society. *American Economic Review* 35(4), 519–530. doi.org/10.2307/1809376.

Cohen, W. M. and Levinthal, D. A. (1990). Absorptive capacity. *Administrative Science Quarterly* 35(1), 128–152. doi.org/10.2307/2393553.

Walsh, J. P. and Ungson, G. R. (1991). Organizational memory. *Academy of Management Review* 16(1), 57–91. doi.org/10.2307/258607.

Garicano, L. (2000). Hierarchies and the organization of knowledge in production. *Journal of Political Economy* 108(5), 874–904. doi.org/10.1086/317673.

Brynjolfsson, E., Li, D. and Raymond, L. R. (2025). Generative AI at work. *Quarterly Journal of Economics* 140(2). doi.org/10.1093/qje/qjae044.

Tools

Obsidian Help, including Canvas and Graph View. help.obsidian.md.

Git documentation. git-scm.com/doc.